

Java 6 — Scripting API

Java Version: 6

Year: 2006

Feature: Scripting API

UniqueClassName: ScriptingAPI

1. Definition

The **Java Scripting API** was introduced in **Java 6 (JDK 1.6)** through **JSR 223 — Scripting for the Java Platform**.

It provides a standard way for a Java application to:

- Execute scripts written in scripting languages.
- Obtain a scripting engine.
- Pass Java variables/objects to scripts.
- Receive values returned by scripts.
- Invoke script functions.
- Allow Java and scripting languages to interact.

The main package is:

`javax.script`

Important classes/interfaces include:

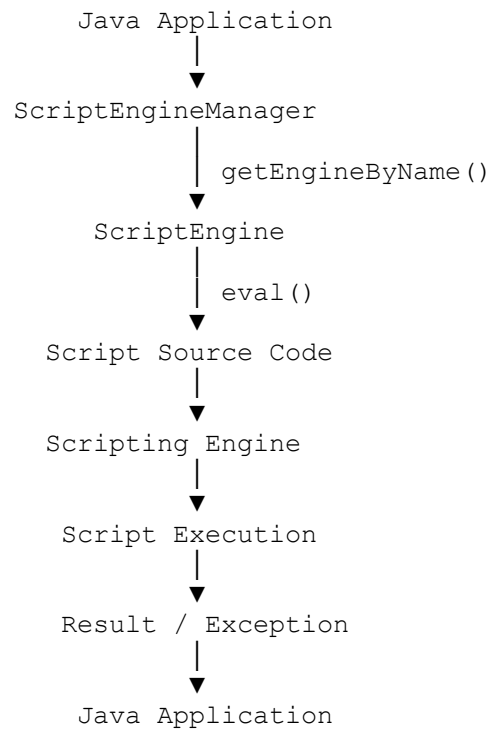
Class / Interface	Purpose
ScriptEngine	Executes scripts
ScriptEngineManager	Finds and creates scripting engines
ScriptEngineFactory	Provides information about a scripting engine
Bindings	Stores variables shared between Java and a script
SimpleBindings	Basic Bindings implementation
ScriptContext	Defines script execution context
Invocable	Allows Java to invoke script functions/methods
Compilable	Allows scripts to be compiled when the engine supports it

Important point

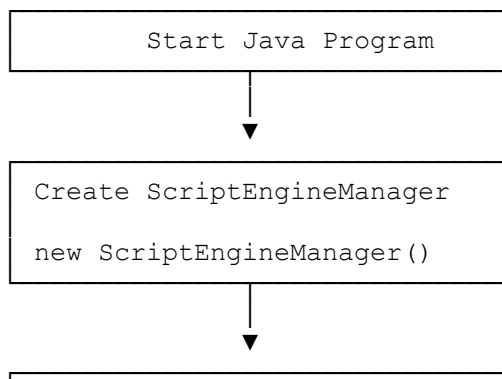
Java 6 provided the **Scripting API**, but the availability of a particular scripting language depends on the scripting engine bundled with or installed for that JDK.

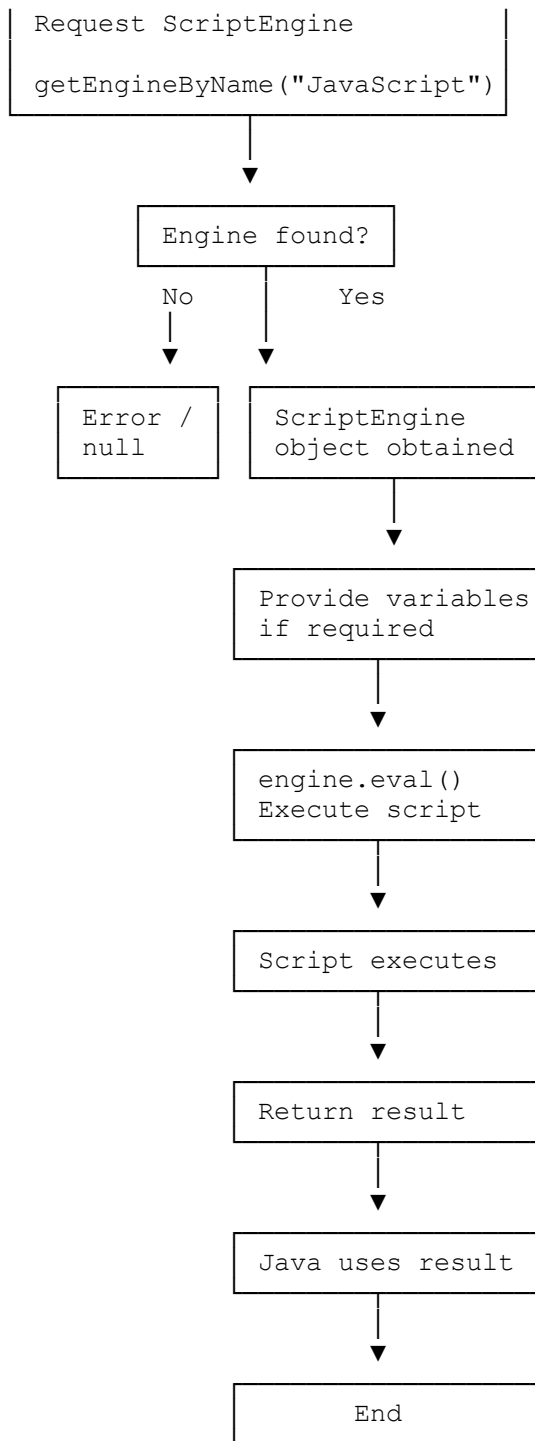
For example, older Java 6 distributions commonly included a **JavaScript scripting engine** based on Rhino. This should not be confused with the modern JavaScript engine situation in current JDKs.

2. Basic Architecture



3. Execution Flowchart





4. Simple Java Code Example

The following example demonstrates Java executing a JavaScript expression through the Java Scripting API.

```
import javax.script.ScriptEngine;
import javax.script.ScriptEngineManager;
import javax.script.ScriptException;

public class ScriptingAPI {

    public static void main(String[] args) {

        try {
            // 1. Create ScriptEngineManager
            ScriptEngineManager manager =
                new ScriptEngineManager();

            // 2. Obtain JavaScript engine
            ScriptEngine engine =
                manager.getEngineByName("JavaScript");

            // 3. Execute script
            Object result =
                engine.eval("10 + 20");

            // 4. Display result
            System.out.println("Result = " + result);

        } catch (ScriptException e) {
            e.printStackTrace();
        }
    }
}
```

Output

Result = 30

5. Java Variable → Script

Java can expose variables to the scripting engine.

```
import javax.script.ScriptEngine;
import javax.script.ScriptEngineManager;
import javax.script.ScriptException;

public class ScriptingAPI {

    public static void main(String[] args) {
```

```

try {
    ScriptEngineManager manager =
        new ScriptEngineManager();

    ScriptEngine engine =
        manager.getEngineByName("JavaScript");

    int a = 10;
    int b = 20;

    engine.put("x", a);
    engine.put("y", b);

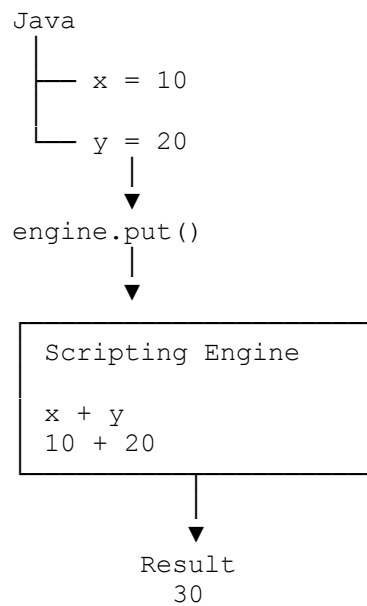
    Object result =
        engine.eval("x + y");

    System.out.println("Result = " + result);

} catch (ScriptException e) {
    e.printStackTrace();
}
}

```

Flow



6. Calling a Script Function from Java

The Invocable interface allows Java to invoke a function defined inside a script.

```
import javax.script.ScriptEngine;
```

```

import javax.script.ScriptEngineManager;
import javax.script.Invocable;

public class ScriptingAPI {

    public static void main(String[] args) throws Exception {

        ScriptEngineManager manager =
            new ScriptEngineManager();

        ScriptEngine engine =
            manager.getEngineByName("JavaScript");

        String script =
            "function add(a, b) {" +
            "    return a + b;" +
            "}";

        engine.eval(script);

        Invocable invocable =
            (Invocable) engine;

        Object result =
            invocable.invokeFunction("add", 10, 20);

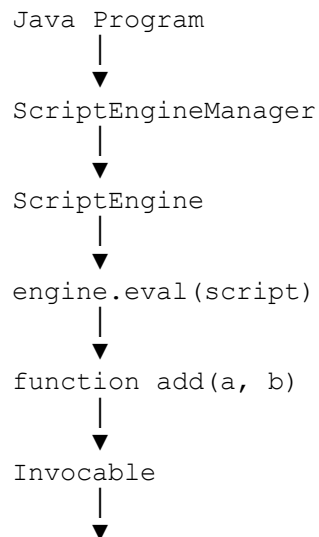
        System.out.println("Result = " + result);
    }
}

```

Output

Result = 30

Execution sequence



```
invokeFunction("add", 10, 20)
```



JavaScript executes



30

7. Using Bindings

Bindings provides a map-like mechanism for sharing variables between Java and the script.

```
import javax.script.*;

public class ScriptingAPI {

    public static void main(String[] args)
        throws ScriptException {

        ScriptEngineManager manager =
            new ScriptEngineManager();

        ScriptEngine engine =
            manager.getEngineByName("JavaScript");

        Bindings bindings = new SimpleBindings();

        bindings.put("name", "Anurag");
        bindings.put("age", 25);

        Object result = engine.eval(
            "'Name: ' + name + ', Age: ' + age",
            bindings
        );

        System.out.println(result);
    }
}
```

Output

Name: Anurag, Age: 25

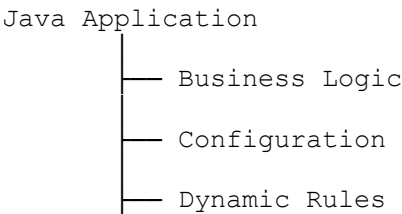
8. Important Classes in Scripting API

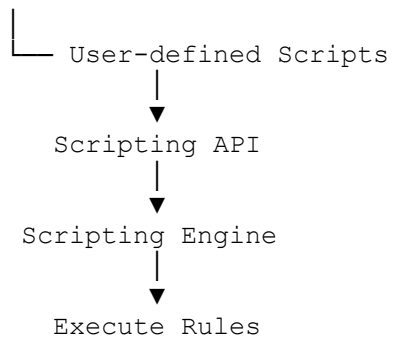
API	Role	Example
ScriptEngineManager	Finds scripting engines	new ScriptEngineManager()
ScriptEngine	Executes scripts	engine.eval()
ScriptEngineFactory	Describes an engine	getEngineName()
Bindings	Shares variables	bindings.put()
SimpleBindings	Basic bindings implementation	new SimpleBindings()
ScriptContext	Controls execution context	engine.setContext()
Invocable	Calls script functions	invokeFunction()
Compilable	Compiles scripts	compile()
CompiledScript	Represents compiled script	eval()

9. eval() vs invokeFunction()

Method	Purpose
engine.eval("10 + 20")	Directly execute script code
engine.eval(script)	Execute a complete script
invocable.invokeFunction("add", ...)	Call a specific script function
compiledScript.eval()	Execute previously compiled script

10. Real-World Use Cases





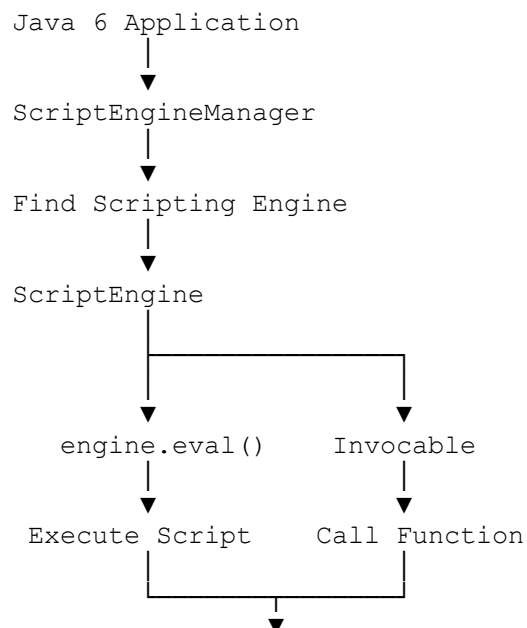
Common applications include:

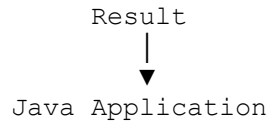
1. **Dynamic business rules** — execute rules without recompiling the main Java application.
2. **Configuration logic** — allow administrators to customize behavior.
3. **Plugin-like scripting** — provide scripting capabilities inside a Java application.
4. **Automation** — execute small scripts from a Java application.
5. **Testing** — dynamically execute expressions or scripts.
6. **Application customization** — allow users to customize application behavior.

11. Java 6 Scripting API in One Line

Java 6 Scripting API provides a standard javax.script framework that allows Java applications to discover scripting engines, execute scripts, exchange data with scripts, and invoke script functions dynamically.

Overall Flow





Key package: javax.script

Introduced: Java 6 / JDK 1.6 (2006)

Specification: JSR 223

Primary interface: ScriptEngine

Primary manager: ScriptEngineManager

Main purpose: Integrate scripting languages with Java applications.

Java 6 — Compiler API

Java Version: 6 / JDK 1.6

Year: 2006

Feature: Compiler API

UniqueClassName: CompilerAPI

1. Definition

The **Compiler API** was introduced in **Java 6 (JDK 1.6)** through **JSR 199 — Java Compiler API**.

It provides a standard Java API that allows a Java application to **programmatically access a Java compiler** and compile Java source code from within the application itself.

The main package is:

javax.tools

The most important interface is:

JavaCompiler

A Java program can therefore perform the basic equivalent of:

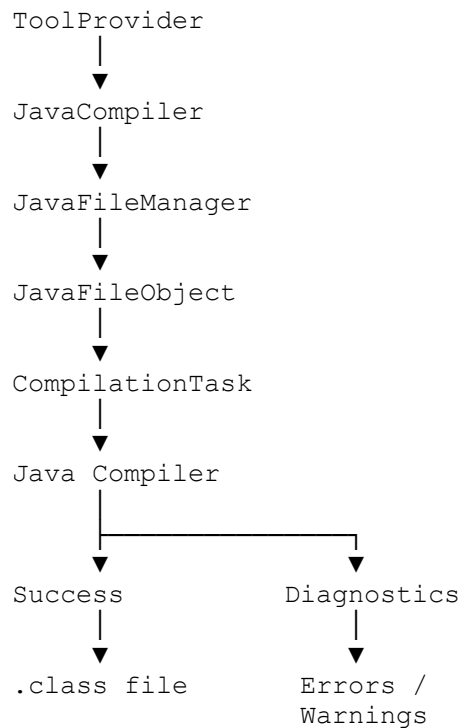
javac MyClass.java

without requiring the user to manually execute the javac command.

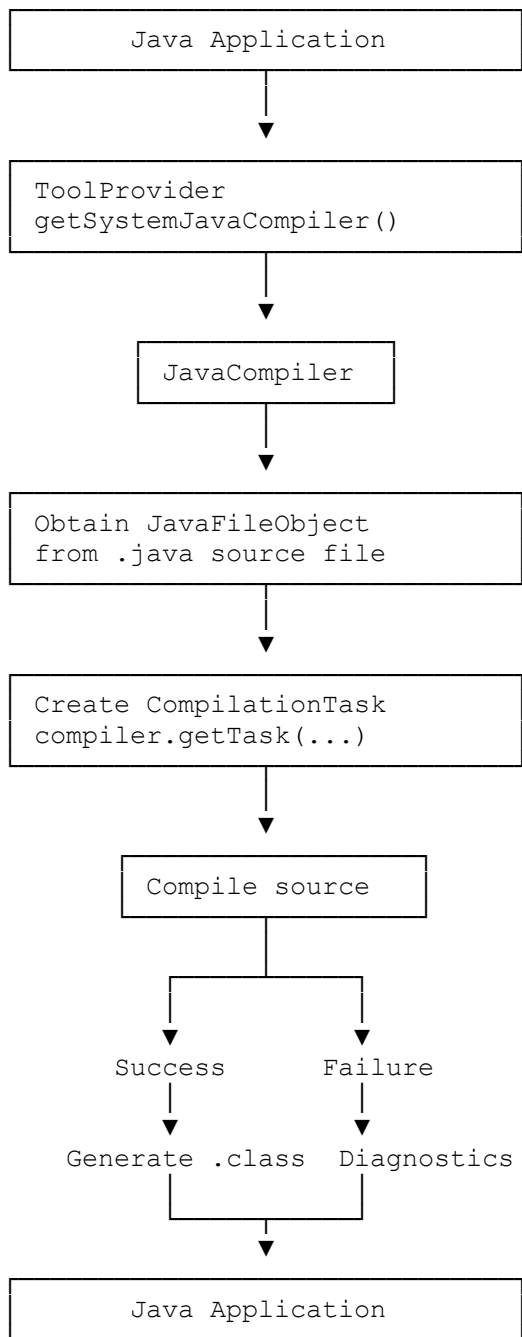
2. Main Components

Component	Purpose
JavaCompiler	Represents the Java compiler
ToolProvider	Provides access to the system Java compiler
JavaFileObject	Represents a Java source/class file
JavaFileManager	Manages source/class files
Diagnostic	Represents compiler errors, warnings, etc.
DiagnosticListener	Receives compiler diagnostics
StandardJavaFileManager	Standard implementation for file management
JavaCompiler.CompilationTask	Represents a compilation task

The basic relationship is:



3. Compiler API Flowchart



4. Simplest Java Code Example

Suppose we have this Java source file:

Hello.java

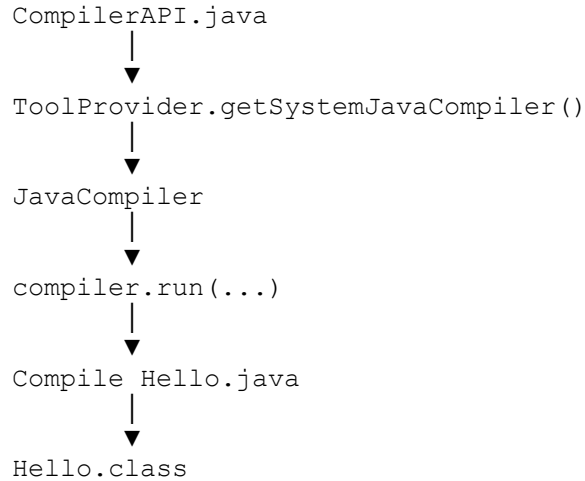
```
public class Hello {  
  
    public static void main(String[] args) {  
        System.out.println("Hello from Compiler API!");  
    }  
}
```

Now another Java program can compile it programmatically.

CompilerAPI.java

```
import javax.tools.JavaCompiler;  
import javax.tools.ToolProvider;  
  
public class CompilerAPI {  
  
    public static void main(String[] args) {  
  
        // Get the system Java compiler  
        JavaCompiler compiler =  
            ToolProvider.getSystemJavaCompiler();  
  
        // Compile Hello.java  
        int result = compiler.run(  
            null,  
            null,  
            null,  
            "Hello.java"  
        );  
  
        // Check compilation result  
        if (result == 0) {  
            System.out.println(  
                "Compilation successful."  
            );  
        } else {  
            System.out.println(  
                "Compilation failed."  
            );  
        }  
    }  
}
```

Execution



If compilation succeeds:

Compilation successful.

5. More Complete Example Using JavaFileManager

The Compiler API also provides a more flexible API through `JavaFileManager`.

```
import javax.tools.JavaCompiler;
import javax.tools.JavaFileObject;
import javax.tools.StandardJavaFileManager;
import javax.tools.ToolProvider;

public class CompilerAPI {

    public static void main(String[] args) {

        JavaCompiler compiler =
            ToolProvider.getSystemJavaCompiler();

        if (compiler == null) {
            System.out.println(
                "Java compiler is not available."
            );
            return;
        }

        StandardJavaFileManager fileManager =
            compiler.getStandardFileManager(
                null,
                null,
                null
            );
    }
}
```

```

        );

        Iterable<? extends JavaFileObject> files =
            fileManager.getJavaFileObjects(
                "Hello.java"
            );

        JavaCompiler.CompilationTask task =
            compiler.getTask(
                null,
                fileManager,
                null,
                null,
                null,
                files
            );

        boolean success = task.call();

        try {
            fileManager.close();
        } catch (Exception e) {
            e.printStackTrace();
        }

        if (success) {
            System.out.println(
                "Compilation successful."
            );
        } else {
            System.out.println(
                "Compilation failed."
            );
        }
    }
}

```

6. Compiler API with Compilation Errors

One major advantage of the API is that a Java application can receive compiler diagnostics.

```

import java.util.Arrays;
import javax.tools.Diagnostic;
import javax.tools.DiagnosticCollector;
import javax.tools.JavaCompiler;
import javax.tools.JavaFileObject;
import javax.tools.StandardJavaFileManager;
import javax.tools.ToolProvider;

public class CompilerAPI {

    public static void main(String[] args) {

```

```

JavaCompiler compiler =
    ToolProvider.getSystemJavaCompiler();

DiagnosticCollector<JavaFileObject>
    diagnostics =
        new DiagnosticCollector<JavaFileObject>();

StandardJavaFileManager fileManager =
    compiler.getStandardFileManager(
        diagnostics,
        null,
        null
    );

Iterable<? extends JavaFileObject> files =
    fileManager.getJavaFileObjects(
        "Hello.java"
    );

JavaCompiler.CompilationTask task =
    compiler.getTask(
        null,
        fileManager,
        diagnostics,
        null,
        null,
        files
    );

boolean success = task.call();

if (success) {
    System.out.println(
        "Compilation successful."
    );
} else {
    System.out.println(
        "Compilation failed."
    );

    for (Diagnostic<? extends JavaFileObject>
        diagnostic : diagnostics.getDiagnostics()) {
        System.out.println(
            "Line: "
            + diagnostic.getLineNumber()
        );

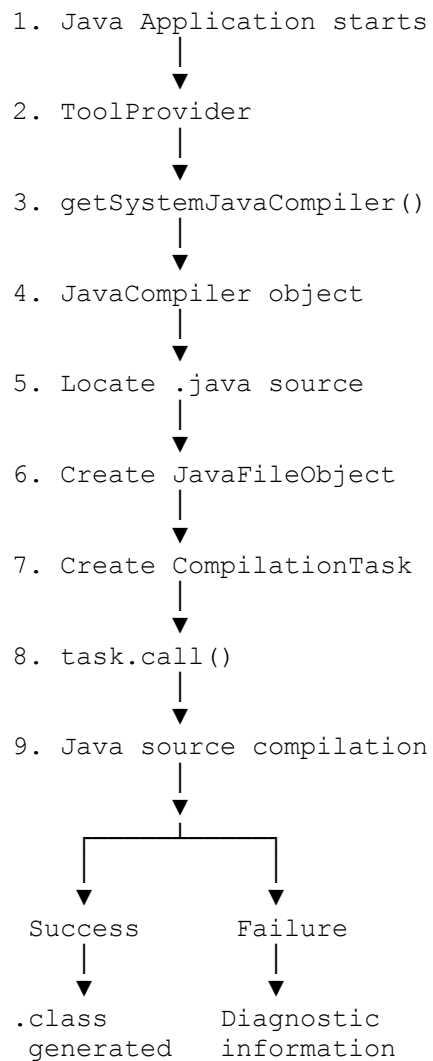
        System.out.println(
            "Message: "
            + diagnostic.getMessage(null)
        );
    }
}

```

```
        try {
            fileManager.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

7. Programmatic Compilation Flow

The internal logical sequence is:



8. compiler.run() vs compiler.getTask()

Feature	compiler.run()	compiler.getTask()
API level	Simple	Advanced
Package	javax.tools	javax.tools
Returns	Exit code	CompilationTask
Diagnostics	Limited	Detailed
File management	Less control	Extensive control
Compiler options	Supported	Supported
Custom file manager	No direct control	Yes
Custom diagnostic handling	Limited	Yes
Suitable for	Simple compilation	IDEs/tools/build systems

Simple approach

```
compiler.run(  
    null,  
    null,  
    null,  
    "Hello.java"  
);
```

Advanced approach

```
JavaCompiler.CompilationTask task =  
    compiler.getTask(...);  
  
boolean success = task.call();
```

9. Important Classes and Interfaces

ToolProvider

Provides access to the system compiler.

```
JavaCompiler compiler =  
    ToolProvider.getSystemJavaCompiler();
```

JavaCompiler

Represents the Java compiler.

```
JavaCompiler compiler =  
    ToolProvider.getSystemJavaCompiler();
```

JavaFileObject

Represents Java source or class-file-like objects.

```
Iterable<? extends JavaFileObject> files =  
    fileManager.getJavaFileObjects(  
        "Hello.java"  
    );
```

CompilationTask

Represents a compilation operation.

```
JavaCompiler.CompilationTask task =  
    compiler.getTask(...);
```

```
boolean success = task.call();
```

Diagnostic

Provides compiler diagnostic information.

```
diagnostic.getLineNumber();  
diagnostic.getMessage(null);
```

10. Important Difference: Compiler API vs javac

javac command	Compiler API
Command-line compiler	Programmatic compiler
Executed from shell/terminal	Executed from Java code
User invokes command	Application invokes compiler
javac Hello.java	compiler.run(...)

Less programmatic control	Extensive programmatic control
Useful for normal development	Useful for Java tools, IDEs, dynamic compilation

Conceptually:

Traditional

```

Developer
  ↓
Terminal
  ↓
javac Hello.java
  ↓
Hello.class

```

Compiler API

```

Java Application
  ↓
ToolProvider
  ↓
JavaCompiler
  ↓
CompilationTask
  ↓
Hello.class

```

11. Important Requirement

`ToolProvider.getSystemJavaCompiler()` obtains the **system Java compiler**. Therefore, when the compiler is not available, it can return null.

For example:

```

JavaCompiler compiler =
    ToolProvider.getSystemJavaCompiler();

if (compiler == null) {
    System.out.println(
        "Compiler is not available."
    );
}

```

}

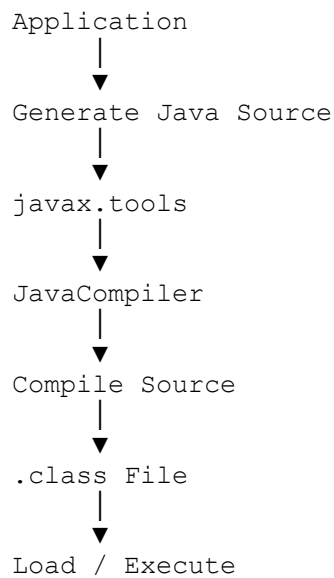
This is particularly relevant when the application is running with a runtime environment that does not contain the compiler.

12. Real-World Applications

The Compiler API is useful when building:

- **IDEs**
- **Java development tools**
- **Build tools**
- **Code generators**
- **Testing frameworks**
- **Dynamic Java code generators**
- **Educational Java compilers/tools**
- **Applications that need to compile generated source code**

For example:



13. Java 6 Compiler API — Summary

Property	Details
Java Version	Java 6 / JDK 1.6
Year	2006
Feature	Compiler API
Specification	JSR 199
Main Package	javax.tools
Main Interface	JavaCompiler
Compiler Provider	ToolProvider
Source Representation	JavaFileObject
Compilation Operation	CompilationTask
Diagnostics	Diagnostic, DiagnosticListener
Main Purpose	Compile Java source programmatically
Typical Output	.class files

In one sentence

Java 6 Compiler API (javax.tools) allows Java applications to programmatically access the Java compiler, compile source code, manage compilation files, and receive compiler diagnostics without directly invoking javac from the command line.

Java 6 — JDBC 4

Java Version: 6 / JDK 1.6

Year: 2006

Feature: JDBC 4.0

UniqueClassName: JDBC4

1. Definition

JDBC 4.0 (Java Database Connectivity 4.0) was introduced with **Java 6 in 2006**.

JDBC 4 provides a standard Java API for connecting Java applications to relational databases, executing SQL statements, processing results, and managing database resources.

The primary package is:

`java.sql`

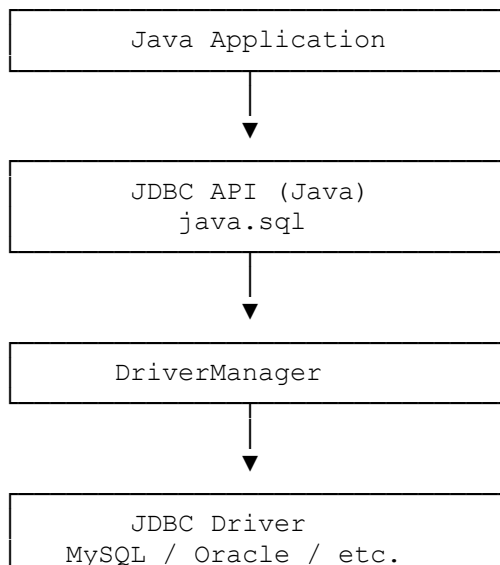
JDBC 4 was specified through **JSR 221 — JDBC 4.0**.

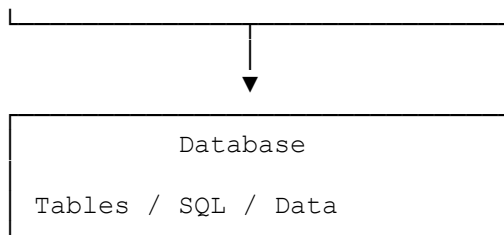
One of the most important improvements in JDBC 4 was **automatic JDBC driver discovery**, which eliminated the need for the traditional explicit:

```
Class.forName("com.example.Driver");
```

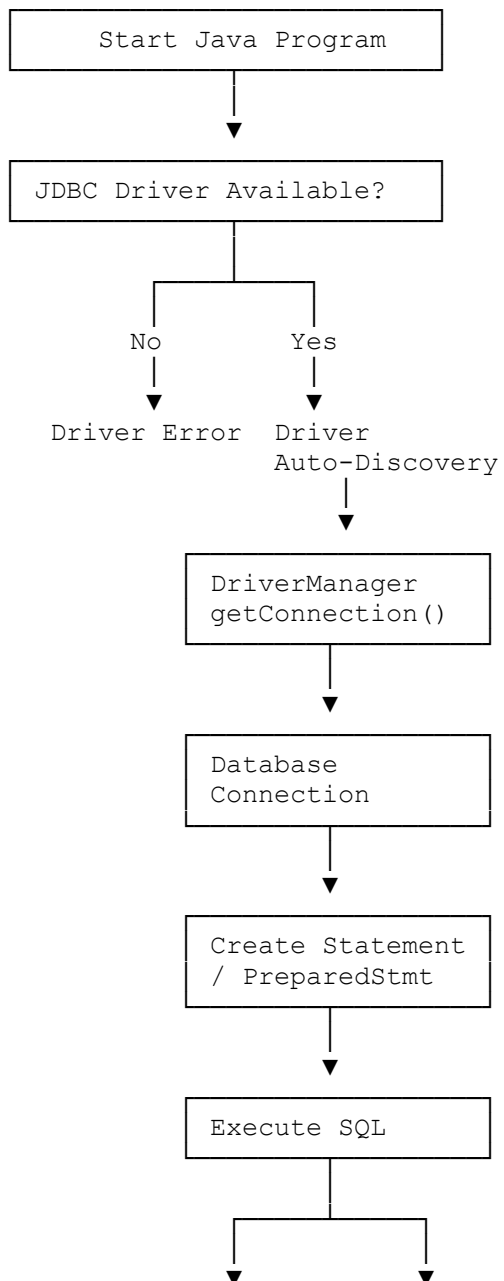
in normal JDBC 4 driver usage, provided the JDBC driver is properly packaged and registered through the service-provider mechanism.

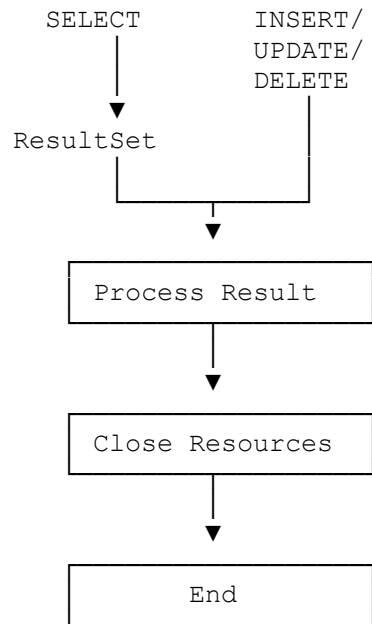
2. JDBC 4 Architecture





3. JDBC 4 Flowchart





4. Main JDBC 4 Components

Component	Purpose
DriverManager	Manages JDBC drivers and creates database connections
Connection	Represents a connection to a database
Statement	Executes simple SQL statements
PreparedStatement	Executes parameterized SQL statements
CallableStatement	Calls stored procedures
ResultSet	Represents query results
SQLException	Represents database/JDBC errors
DatabaseMetaData	Provides information about the database
ResultSetMetaData	Provides information about query-result columns
Driver	Defines the JDBC driver interface

5. Basic JDBC 4 Java Code Example

The following example uses MySQL conceptually.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class JDBC4 {

    public static void main(String[] args) {

        String url =
            "jdbc:mysql://localhost:3306/testdb";

        String username = "root";
        String password = "password";

        try (
            Connection connection =
                DriverManager.getConnection(
                    url,
                    username,
                    password
                );

            Statement statement =
                connection.createStatement();

            ResultSet resultSet =
                statement.executeQuery(
                    "SELECT id, name FROM employee"
                )
        ) {

            while (resultSet.next()) {

                int id =
                    resultSet.getInt("id");

                String name =
                    resultSet.getString("name");

                System.out.println(
                    id + " : " + name
                );

            }

        } catch (Exception e) {
            e.printStackTrace();
        }

    }

}
```

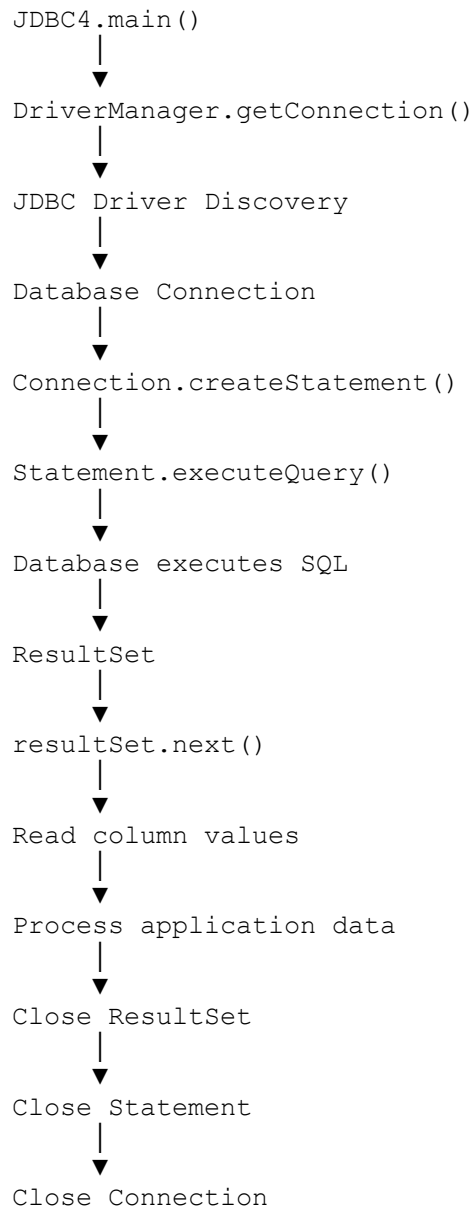
Important JDBC 4 point

Notice that there is **no explicit**:

```
Class.forName("com.mysql.jdbc.Driver");
```

JDBC 4 introduced automatic driver loading through the JDBC driver discovery mechanism.

6. JDBC 4 Execution Sequence



7. JDBC 4 with PreparedStatement

For SQL containing input values, PreparedStatement is generally preferable to building SQL strings manually.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

public class JDBC4 {

    public static void main(String[] args) {

        String url =
            "jdbc:mysql://localhost:3306/testdb";

        String username = "root";
        String password = "password";

        String sql =
            "INSERT INTO employee(id, name) VALUES (?, ?)";

        try (
            Connection connection =
                DriverManager.getConnection(
                    url,
                    username,
                    password
                );

            PreparedStatement statement =
                connection.prepareStatement(sql)
        ) {

            statement.setInt(1, 101);
            statement.setString(2, "Anurag");

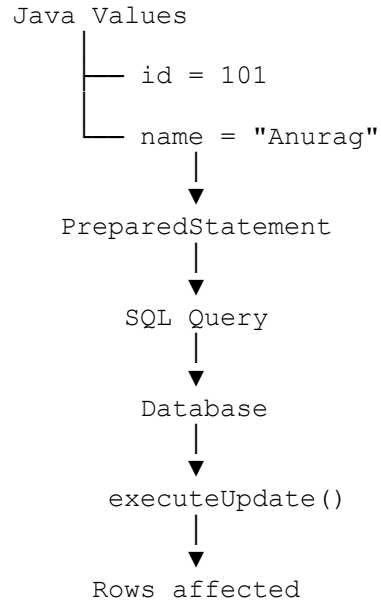
            int rows =
                statement.executeUpdate();

            System.out.println(
                "Rows inserted: " + rows
            );

        } catch (Exception e) {
            e.printStackTrace();
        }

    }
}
```

Flow



8. JDBC 4 Automatic Driver Loading

Traditional JDBC approach

Before JDBC 4, applications commonly explicitly loaded a JDBC driver:

```
Class.forName(  
    "com.mysql.jdbc.Driver"  
);
```

Then:

```
Connection connection =  
    DriverManager.getConnection(  
        url,  
        username,  
        password  
    );
```

JDBC 4 approach

With a JDBC 4-compatible driver:

```
Connection connection =  
    DriverManager.getConnection(  
        url,
```

```
    username,  
    password  
);
```

The driver can be discovered automatically through the JDBC service-provider mechanism.

JDBC 3 and earlier

```
Application  
  |  
  ▼  
Class.forName()  
  |  
  ▼  
Load Driver  
  |  
  ▼  
DriverManager  
  |  
  ▼  
Database
```

JDBC 4

```
Application  
  |  
  ▼  
DriverManager  
  |  
  ▼  
Automatic Driver Discovery  
  |  
  ▼  
JDBC Driver  
  |  
  ▼  
Database
```

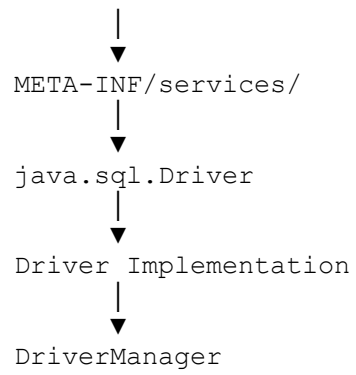
Important: The JDBC driver itself still has to be present on the application's classpath/module path as appropriate. JDBC 4 removes the usual need for the application's source code to explicitly call `Class.forName()`; it does **not** eliminate the need for a driver.

9. JDBC 4 and ServiceLoader

The automatic discovery mechanism is based on the JDBC driver's service-provider registration.

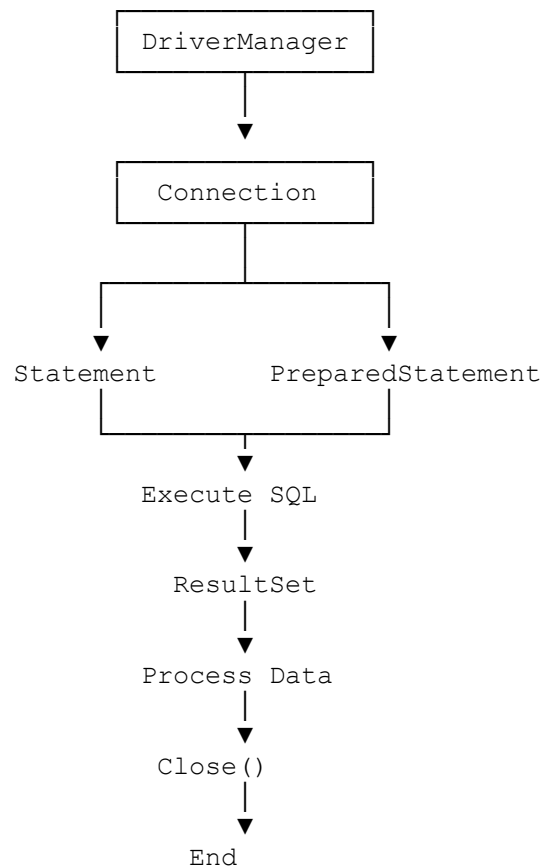
Conceptually:

JDBC Driver JAR



This allows DriverManager to discover JDBC drivers without application code explicitly loading each driver class.

10. JDBC 4 Connection Lifecycle



11. JDBC 4 vs Earlier JDBC

Feature	JDBC 3	JDBC 4
Java Version	Java 1.4/5 era	Java 6
Year	2000s	2006
Specification	JSR 54	JSR 221
Package	java.sql	java.sql
Automatic driver discovery	✗	✓
Class.forName() normally required	Commonly used	Usually unnecessary
SQLXML support	✗	✓
Improved exception handling	Limited	Enhanced
SQLException chaining	Limited	Improved
RowId support	✗	✓
SQLData/advanced SQL type support	Existing	Enhanced
Better integration with Java 6	✗	✓

12. JDBC 4 Major Improvements

1. Automatic Driver Discovery

The biggest commonly noticed change:

```
DriverManager.getConnection(...)
```

can automatically discover a JDBC 4 driver.

2. Better Exception Handling

JDBC 4 introduced improvements around SQL exceptions, including enhanced support for chained exceptions.

3. SQLXML

JDBC 4 added standard support for SQL XML data through:

SQLXML

4. RowId

JDBC 4 introduced:

RowId

for database row identifiers where supported by the database.

5. Better Resource Handling

Java 6 also introduced language/library improvements that made JDBC resource management easier to write, although **try-with-resources itself is a Java 7 feature**, not a JDBC 4/Java 6 feature.

Therefore, if writing code specifically for **Java 6**, resources should generally be closed explicitly in a finally block.

13. Java 6-Compatible JDBC 4 Example

Because Java 6 does **not** support try-with-resources, the Java 6 style is:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

public class JDBC4 {

    public static void main(String[] args) {

        Connection connection = null;
        PreparedStatement statement = null;

        try {

            String url =
                "jdbc:mysql://localhost:3306/testdb";

            connection =
                DriverManager.getConnection(
                    url,
                    "root",
                    "password"
                );

            statement =
```

```

        connection.prepareStatement(
            "INSERT INTO employee " +
            "(id, name) VALUES (?, ?)"
        );

        statement.setInt(1, 101);
        statement.setString(2, "Anurag");

        int rows =
            statement.executeUpdate();

        System.out.println(
            "Rows inserted: " + rows
        );
    } catch (Exception e) {

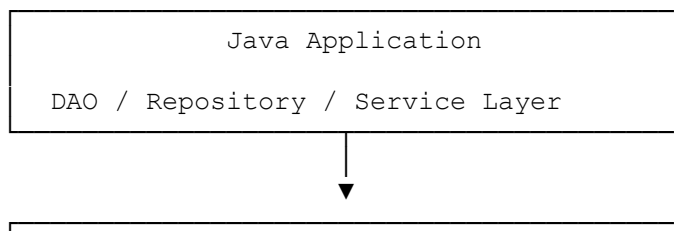
        e.printStackTrace();
    } finally {

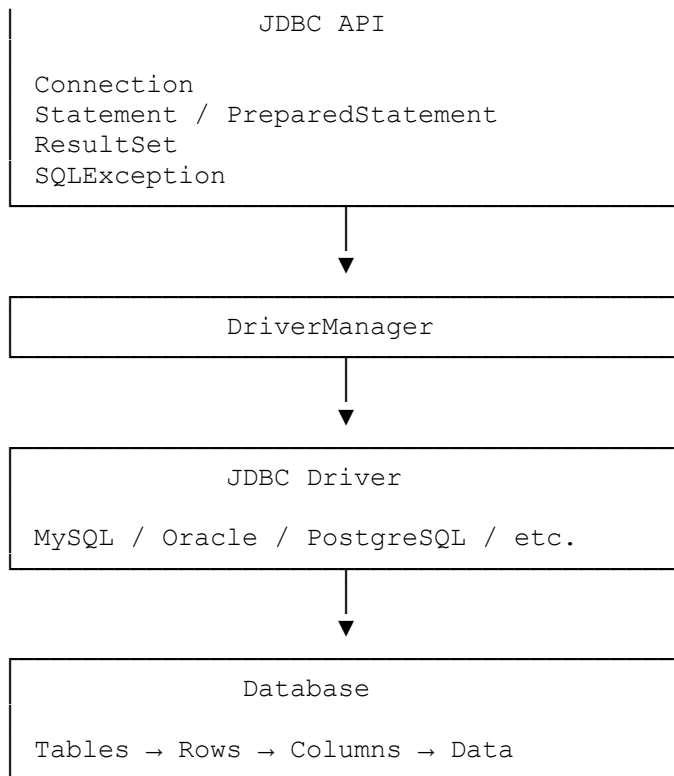
        try {
            if (statement != null) {
                statement.close();
            }
        } catch (Exception e) {
            e.printStackTrace();
        }

        try {
            if (connection != null) {
                connection.close();
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

14. Real-World JDBC 4 Architecture





15. Summary

Property	Details
Java Version	Java 6 / JDK 1.6
Year	2006
Feature	JDBC 4.0
Specification	JSR 221
Main Package	java.sql
Main Entry Point	DriverManager
Connection API	Connection
SQL API	Statement, PreparedStatement, CallableStatement
Query Result	ResultSet
Major Improvement	Automatic JDBC driver discovery

Other Improvements	SQLXML, RowId, improved exception handling
Explicit Class.forName()	Normally unnecessary with JDBC 4 drivers
Try-with-resources	Not part of Java 6; introduced in Java 7
Primary Purpose	Standard Java-to-database connectivity

In one sentence

JDBC 4.0, introduced in Java 6 (2006), modernized Java database connectivity by providing automatic JDBC driver discovery and enhanced APIs for SQL/XML, row identifiers, exceptions, and database interaction through the standard java.sql package.

Java 6 — Web Services APIs (JAX-WS)

Java Version: 6 / JDK 1.6

Year: 2006

Feature: Web Services APIs (JAX-WS)

UniqueClassName: WebServicesJAXWS

1. Definition

JAX-WS (Java API for XML Web Services) is a Java API for developing and consuming **XML-based web services**, particularly **SOAP-based web services**.

JAX-WS became part of the standard Java SE platform in **Java 6 (JDK 1.6)**. It replaced the older **JAX-RPC** programming model.

The main packages include:

```
javax.xml.ws
javax.jws
```

JAX-WS allows a Java application to expose Java methods as web-service operations and allows another application to call those operations remotely.

Basic concept

```
Java Method
↓
```

JAX-WS
↓
SOAP/XML Message
↓
HTTP
↓
Remote Web Service

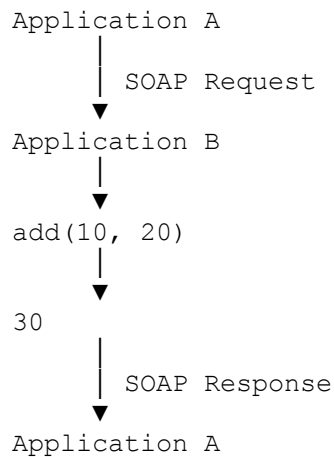
2. What Problem Does JAX-WS Solve?

Suppose one application has:

`add(10, 20)`

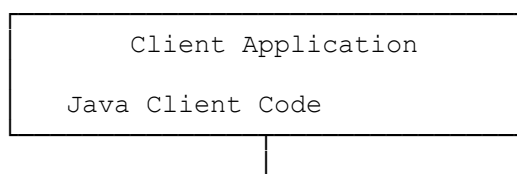
and another application is running on a different computer.

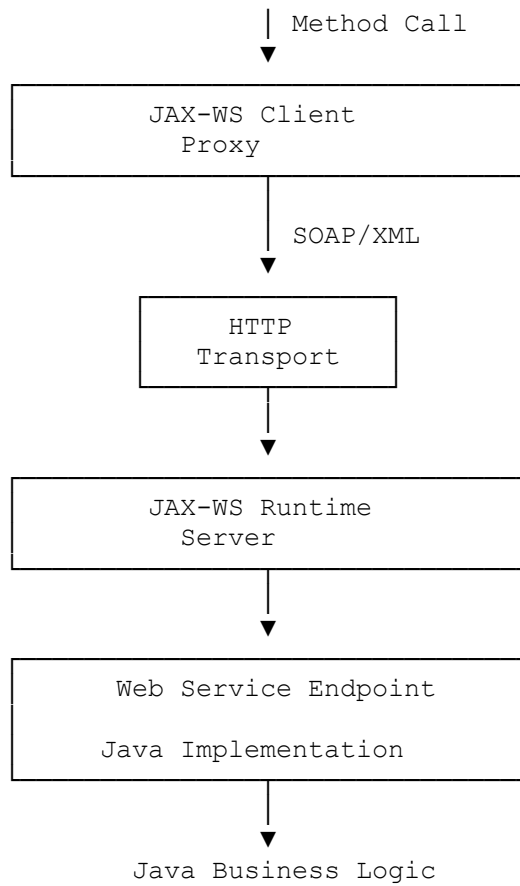
Instead of directly calling the Java method, JAX-WS can expose that method as a web-service operation.



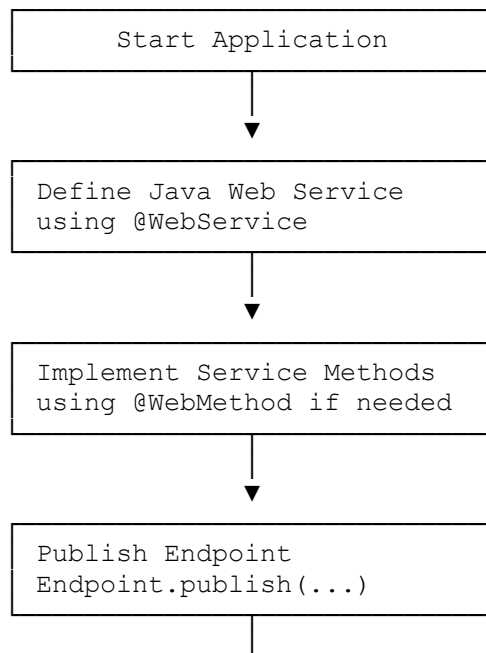
The two applications do not need to be running inside the same JVM.

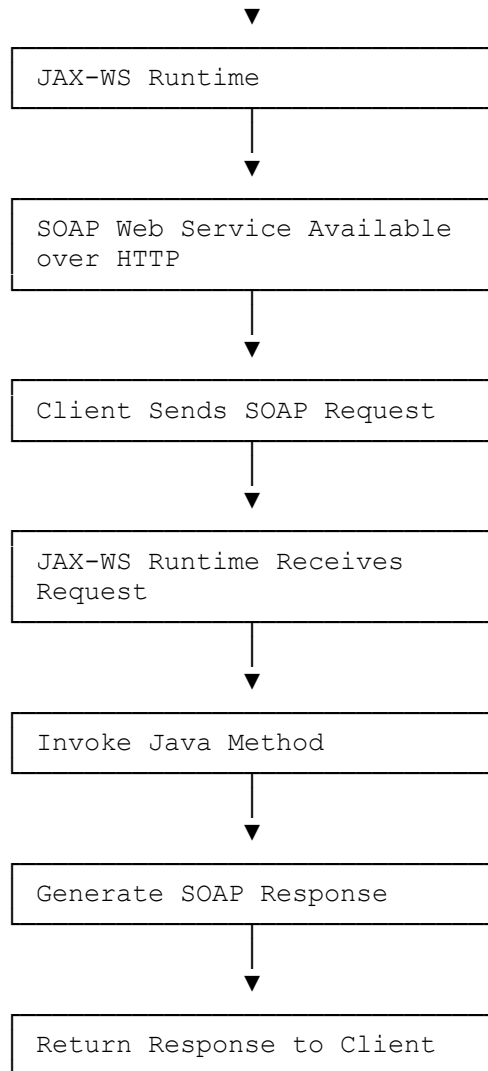
3. JAX-WS Architecture





4. JAX-WS Flowchart





5. Important JAX-WS Annotations and Classes

API	Purpose
@WebService	Marks a Java class/interface as a web service
@WebMethod	Exposes a method as a web-service operation
@WebParam	Defines a web-service parameter
@WebResult	Defines the web-service return value
Endpoint	Publishes a web-service endpoint
Service	Represents a client-side web-service

WebServiceClient	Identifies generated service-client classes
WebServiceEndpoint	Represents web-service endpoint information
BindingProvider	Provides access to client request context
SOAPBinding	Provides SOAP binding information

6. Simple JAX-WS Web Service

Step 1 — Create the Web Service

```
import javax.jws.WebMethod;
import javax.jws.WebService;

@WebService
public class WebServicesJAXWS {

    @WebMethod
    public String hello(String name) {
        return "Hello, " + name;
    }
}
```

The important annotation is:

```
@WebService
```

It tells the JAX-WS runtime that the class represents a web-service endpoint.

7. Publishing the Web Service

In Java 6, the Endpoint API can be used to publish a simple endpoint.

```
import javax.xml.ws.Endpoint;

public class WebServicesJAXWS {

    public String hello(String name) {
        return "Hello, " + name;
    }

    public static void main(String[] args) {
        Endpoint.publish(
```

```

        "http://localhost:8080/hello",
        new WebServicesJAXWS()
    );

    System.out.println(
        "Web service started."
    );
}
}

```

For a more explicit service declaration:

```

import javax.jws.WebMethod;
import javax.jws.WebService;
import javax.xml.ws.Endpoint;

@WebService
public class WebServicesJAXWS {

    @WebMethod
    public String hello(String name) {
        return "Hello, " + name;
    }

    public static void main(String[] args) {

        Endpoint endpoint =
            Endpoint.publish(
                "http://localhost:8080/hello",
                new WebServicesJAXWS()
            );

        System.out.println(
            "Service published."
        );
    }
}

```

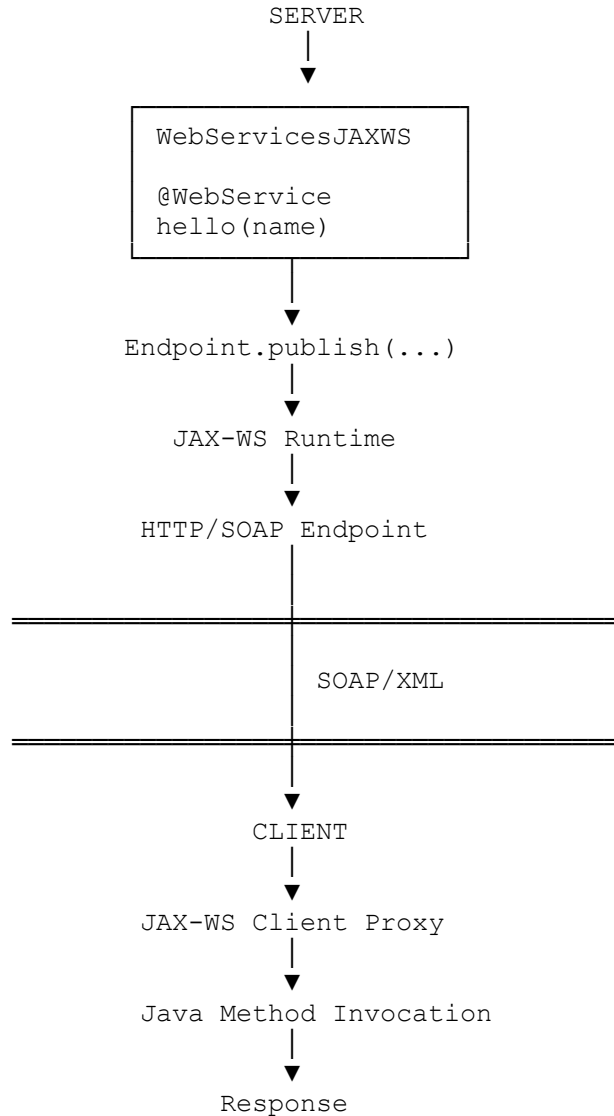
The endpoint becomes conceptually available at:

<http://localhost:8080/hello>

and the WSDL is commonly exposed at:

<http://localhost:8080/hello?wsdl>

8. JAX-WS Complete Execution Flow



9. SOAP Message Concept

When the client invokes:

`hello("Anurag")`

JAX-WS can represent the request using SOAP/XML.

Conceptually:

```
<soap:Envelope>  
  <soap:Body>
```

```
<hello>
  <name>Anurag</name>
</hello>
</soap:Body>
</soap:Envelope>
```

The server processes the request and produces a SOAP response conceptually similar to:

```
<soap:Envelope>
  <soap:Body>
    <helloResponse>
      <return>Hello, Anurag</return>
    </helloResponse>
  </soap:Body>
</soap:Envelope>
```

The application developer normally does **not** have to manually construct these SOAP messages when using the standard JAX-WS programming model.

10. Client-Side Concept

A JAX-WS client can use a generated proxy to communicate with the service.

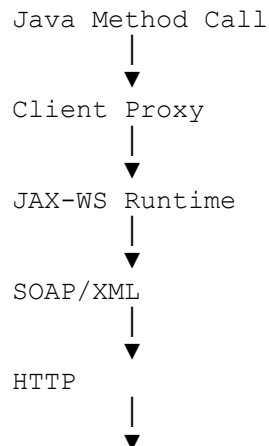
Conceptually:

```
WebServicePort port = ...;
```

```
String result =
    port.hello("Anurag");
```

```
System.out.println(result);
```

The proxy hides the underlying SOAP/XML communication.

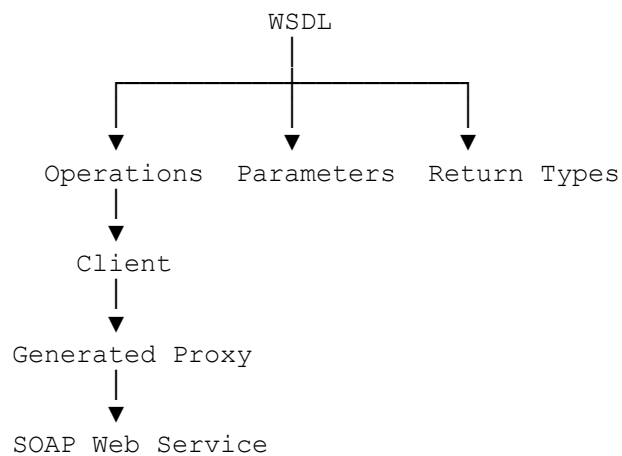


Server
↓
Java Method

11. WSDL in JAX-WS

WSDL (Web Services Description Language) describes the web service contract.

Conceptually:



A WSDL can describe:

- Available operations
 - Input parameters
 - Output values
 - Data types
 - Service endpoint
 - SOAP bindings
-

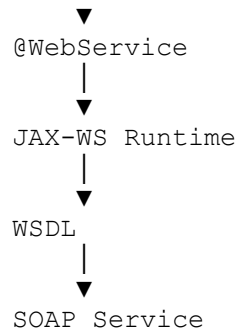
12. Code-First vs Contract-First

JAX-WS supports both common approaches.

Code-First

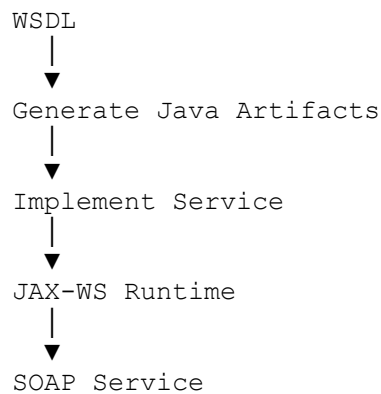
Start with Java:

Java Class
|



Contract-First

Start with WSDL:



13. Important JAX-WS Java 6 APIs

@WebService

```
@WebService
public class Calculator {
}
```

Declares a web-service class.

@WebMethod

```
@WebMethod
public int add(int a, int b) {
    return a + b;
}
```

Exposes a method as a web-service operation.

Endpoint

```
Endpoint.publish(
    "http://localhost:8080/calculator",
    new Calculator()
);
```

Publishes the service.

Service

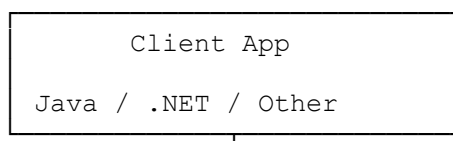
The client-side Service API represents a web-service client service and can be used with generated service artifacts.

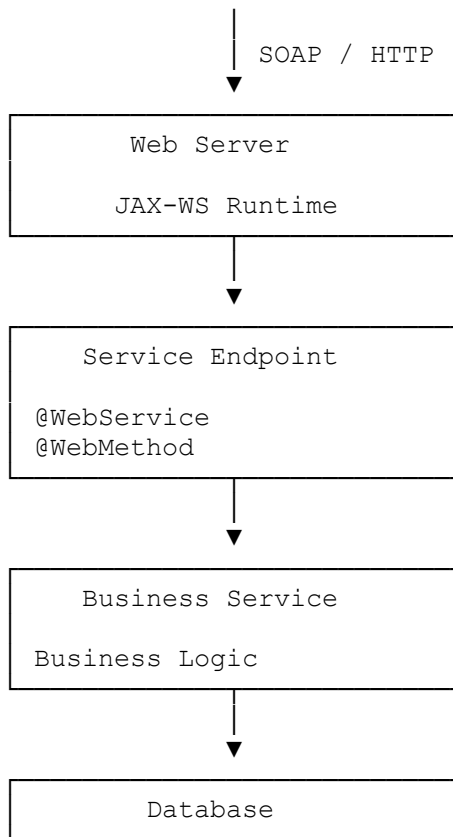
14. JAX-WS vs JAX-RPC

Feature	JAX-RPC	JAX-WS
Purpose	SOAP web services	SOAP/XML web services
Generation era	Older	Newer
Java 6	Superseded	Standard API
Annotation-based programming	Limited	Strong support
@WebService	✗	✓
@WebMethod	✗	✓
Endpoint API	✗	✓
Ease of development	Lower	Higher
Primary standard	JAX-RPC	JAX-WS

15. Real-World Architecture

A typical enterprise architecture can look like:





16. Java 6 JAX-WS Summary

Property	Details
Java Version	Java 6 / JDK 1.6
Year	2006
Feature	JAX-WS
Full Name	Java API for XML Web Services
Specification	JAX-WS 2.0 / JSR 224
Main Packages	javax.xml.ws, javax.jws
Primary Protocol	SOAP
Typical Transport	HTTP
Main Annotation	@WebService

Method Annotation	@WebMethod
Endpoint API	Endpoint
Description Language	WSDL
Replaced/Superseded	JAX-RPC programming model
Main Purpose	Build and consume SOAP/XML web services

Important version detail: JAX-WS 2.0 was included in **Java SE 6**, while the broader JAX-WS specification has had later revisions. Also, JAX-WS is specifically associated with **SOAP/XML web services**; it should not be confused with REST APIs such as JAX-RS.

In one sentence

JAX-WS in Java 6 provides a standard annotation-based and runtime-based API for exposing Java methods as SOAP web-service operations and for consuming remote SOAP services through Java clients.